

AIR SENTINEL

Remaining Team Work Guide

A step-by-step plan to finish AirSentinel together

Purpose

This guide tells the team exactly what still needs people, accounts, approvals and deployment. Start at Step 1 and do not claim an external integration is live until its checkpoint passes.

What is inside

- A clear ownership split
- Data improvement work
- Google Cloud set-up and deployment
- GitHub, video and submission steps
- Definitions of done and troubleshooting

Prepared from the verified local AirSentinel repository. It clearly separates what already works from work that needs your team's accounts, approvals or field validation.

1. What is already complete

The local prototype works: public, citizen and authority dashboards; a private FastAPI backend; six-pollutant data contracts; locality evidence gates; one-, three- and four-hour historical forecast candidates; anomaly review; audit records; feature-flagged Google adapters; deployment scripts; and 38 passing tests.

Important truth

The project is not yet a live government system. OpenAQ is a labelled development fallback, current CPCB/data.gov.in access is not configured, and Google Cloud cannot be deployed until your team uses its own project, billing and identities.

2. Give each person one clear owner role

Role	Owns	Finish line
Project/cloud owner	GCP project, billing, region, IAM, Secret Manager, Cloud Run	Public/private URLs work and access is least privilege.
Data owner	CPCB/data.gov.in access, official archive requests, station mapping, units and calibration	Every data source has provenance and locality relationship evidence.
ML owner	Coverage gate, chronological baselines, uncertainty and drift checks	Najafgarh/Safdarjung are evaluated without lowering safety thresholds.
Citizen/governance owner	Consent, Firebase Auth, moderation, retention and reviewer roles	Reports cannot be mistaken for official measurements.
Demo/release owner	GitHub, deployed links, screenshots, pitch, 3–5 minute video	Judges can evaluate the whole flow end to end.

3. Work in this order

Do not skip this sequence

- 1. Create ownership** — Name the five owners above and put the shared project/contact details in a team document.
- 2. Secure data** — Request and document the approved current/historical source before treating the model as operational.
- 3. Configure cloud** — Create one team-owned GCP project, enable billing and add restricted service identities.
- 4. Deploy safely** — Deploy public dashboard separately from private API and authority dashboard.
- 5. Test identities** — Test allowed and denied actions, persistence and audit logs.
- 6. Package proof** — Push GitHub, record video and submit only verified URLs and claims.

4. Data improvement: the highest-priority team task

The system requires six outdoor pollutants: PM2.5, PM10, NO₂, O₃, SO₂ and CO. The preferred production path is an authorised Indian source such as CPCB/data.gov.in or an approved CPCB/SPCB/PCC export. The current long history is a clearly marked OpenAQ development fallback; it is useful for research, not proof of official operational readiness.

Locality	Available fallback history	What still needs validation
Najafgarh pilot area	7,516 PM2.5 rows; 181.9 days; 86.1% completeness; two stations	Approved boundary, station-to-locality mapping, calibration/uptime and a controlled baseline evaluation.
Safdarjung Enclave pilot area	7,889 PM2.5 rows; 181.9 days; 90.4% completeness; two stations	Approved boundary, station-to-locality mapping, calibration/uptime and a controlled baseline evaluation.

1. Ask CPCB/SPCB/PCC or the authorised data.gov.in contact for current observations, a historical export, station metadata, units, calibration/maintenance and uptime information.

2. Create a station mapping sheet: station ID, coordinates, source, operating status, distance to locality, whether it is inside the approved locality boundary, and responsible authority.
3. Keep the phrase 'nearby-station proxy' until an authority approves the locality geometry and physical station relationship.
4. Store approved analytical tables only. Never commit raw downloads, personal reports, credentials or audit data to GitHub.
5. Re-run the repository coverage and promotion checks after new data arrives. Do not lower the 240 safe-row threshold simply to get a result.

```
cd C:\Users\Rupesh\AirSentinel\AirSentinel
```

```
.\.venv\Scripts\python.exe src\data\report_multiscale_coverage.py
```

```
.\.venv\Scripts\python.exe src\models\report_model_promotion_readiness.py
```

Current model gate

Najafgarh has 224/207/202 safe aligned evaluation rows for 1h/3h/4h; Safdarjung has 151/135/131. The controlled requirement is 240 per horizon, so both remain historical prototype candidates rather than promoted local models.

5. Google Cloud work: complete this as one team-owned setup

Use one Google Cloud project owned by the team, not a personal demo project. Choose the region, owner, data retention policy and billing owner before deploying. Keep secrets in Secret Manager, never in a source file or screenshot.

Service	Why AirSentinel uses it	Team action
Cloud Run	Hosts public dashboard separately from private services.	Create project/billing; deploy and test each URL.
Firebase	Verified citizen identity and pending-review evidence.	Enable Auth, apply deny-by-default rules, set consent/retention.

Service	Why AirSentinel uses it	Team action
BigQuery	Approved analytics/audit reporting at scale.	Create dataset and least-privilege writer/viewer roles.
Gemini	Bounded explanation and photo evidence triage.	Enable API, set key/identity and keep human review.
Maps / Earth Engine	Map context and satellite context—not source proof.	Restrict Maps key; register Earth Engine project.
Translation / STT / TTS	Accessibility for consented citizen reports.	Enable only if required, test target languages and consent text.

Safe deployment checklist

6. Read deploy/README_GOOGLE_CLOUD.md together before running a command.
7. Create/select the team GCP project. Enable billing and record its ID in the team password manager/document—not in public code.
8. Install and authenticate gcloud on the cloud owner's computer. Use separate least-privilege identities for deployment and runtime.
9. Run the repository bootstrap scripts with the real project ID. Review every resource before allowing it to be created.
10. Add reviewed secret values directly in Google Secret Manager. Do not use .env values in Cloud Run command arguments.
11. Deploy three separated surfaces: public aggregate dashboard, private API, and private authority dashboard.
12. Test one citizen identity, one reviewer identity, one denied/unprivileged user, audit persistence and service restart behaviour.

```
Get-Content deploy\README_GOOGLE_CLOUD.md
```

```
powershell -ExecutionPolicy Bypass -File deploy\gcp-bootstrap.ps1 -ProjectId <YOUR_PROJECT_ID>
```

```
powershell -ExecutionPolicy Bypass -File deploy\cloud-run-deploy.ps1 -ProjectId <YOUR_PROJECT_ID>
-Region <YOUR_REGION>
```

Do not invent a production action

Cloud Functions, Dialogflow and any external notification are optional until the hackathon rules and an approved event contract give you an owner, consent policy and idempotent trigger. The existing system correctly avoids pretending these are live.

6. Deployment acceptance tests

Test	Expected result	Pass condition
Public dashboard	Only aggregate, allowlisted information is shown.	No case action, raw report or contact data is exposed.
Citizen report	Verified user can submit coarse, consented evidence.	Submission enters pending review; it is not treated as a sensor measurement.
Authority action	Verified reviewer can view and close a case.	The action is durable and audit-log fields are stored.
Negative access	Unauthorised user is refused.	Private endpoints and Firestore/Storage client access deny by default.
Failure handling	A disabled Google adapter returns a clear safe message.	No fake Gemini/Maps/Earth Engine result is shown.

7. GitHub and submission package

13. Create a new GitHub repository under a team-owned account or organisation.
14. Run the tests and inspect the status before the first commit. Ensure .env, service-account files, raw data, citizen reports, audit records and model binaries are excluded.
15. Push only the curated source tree and approved documentation. The repository currently has a curated submission ZIP but GitHub publication and remote setup are team work.
16. Record the 3–5 minute video using docs/DEMO_VIDEO_SCRIPT.md and docs/DEMO_RUNBOOK.md. Show a real end-to-end flow, its safety boundaries and the final deployed link.

17. Open docs/SUBMISSION_CHECKLIST.md and tick every external link, team name and required form field only after verification.

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

```
git status
```

```
git add .
```

```
git commit -m \"Prepare AirSentinel hackathon submission\"
```

Before pressing submit

The source URL, demo URL, deployed public URL, team details and 3–5 minute video must be real and tested. Do not say ‘deployed’, ‘official’, ‘live alert’ or ‘source identified’ unless your evidence supports it.

8. A two-day finishing plan

When	Owner	Deliverable
Day 1 morning	Cloud + data	GCP project/billing/roles; official-source request; station mapping sheet.
Day 1 afternoon	Cloud + governance	Firebase/BigQuery/secrets configured; consent, roles and retention reviewed.
Day 2 morning	Cloud + QA	Cloud Run deployments; positive/negative identity tests; audit persistence test.
Day 2 afternoon	Release + demo	GitHub push, video recording, pitch/deployed links and submission form review.

9. When you are genuinely done

- The deployed public view is reachable and excludes sensitive/authority-only material.
- The authority view and API reject non-reviewer users, and every reviewed action is audited durably.
- Official data status, fallback status, freshness, locality relationship and uncertainty are visible.
- Your team can explain that a sudden spike becomes a reviewed case, not an automatic accusation or enforcement event.
- GitHub, demo video, pitch deck, short description and deployed link are all included and checked.